杭州电子科技大学

《计算机网络实验》实验报告

课程号：(2023-2024-1)-S0507170-09

实验二　HTTP 抓包及分析

指导老师：

姓名学号：

2026 年 11 月

一、实验目的

- 通过实际抓包观察 HTTP 请求报文与响应报文的格式，掌握请求行、状态行、首部字段与实体主体的具体构成；
- 理解 HTTP/1.1 的持续连接与非持续连接的区别，掌握条件 GET（Conditional GET）的缓存验证机制；
- 掌握带内嵌对象的 Web 页面加载过程中多个 HTTP 请求的时序关系，理解"串行下载"与"并行下载"的差异；
- 理解 HTTP 基本认证（Basic Authentication）的质询—应答过程，并认识 Base64 编码并不等于加密；
- 熟悉 Wireshark 的 HTTP 协议解析与"跟随 TCP 流"功能。

二、实验设备和环境

表 1 实验软硬件环境

项目	配置说明
客户端主机	Windows 11，IPv4 192.168.1.3（WLAN 网卡），临时端口由系统在 49152–65535 范围内分配
服务器	gaia.cs.umass.edu，IPv4 128.119.245.12，TCP 端口 80
访问方式	curl 8.22.0 命令行（精确控制请求头）＋ Chrome 浏览器（获取真实的浏览器请求头并观察并行下载）
抓包工具	Wireshark 3.6.16 / dumpcap，抓包网卡 \Device\NPF_{EB84B5BB-...}
捕获过滤器	tcp port 80 and host 128.119.245.12
捕获文件	cap_http.pcapng（curl，62 个报文）、cap_http2.pcapng（Chrome，含并行下载）

三、实验过程及步骤

3.1 抓包与流量隔离

本机所在网络中同时存在夸克浏览器更新、微信、Windows 证书吊销列表下载等后台流量，若不加过滤，抓到的报文中 HTTP 部分会被大量 TLS 与 TCP 报文淹没。因此先用捕获过滤器把范围限定为"与实验服务器 128.119.245.12 之间的 80 端口流量"：

```
> "E:\tools\Wireshark\dumpcap.exe" -i 6 -f "tcp port 80 and host 128.119.245.12" \
      -a duration:40 -w cap_http.pcapng
```

捕获期间依次用 curl 访问实验站点上的五个页面，覆盖了页面正常返回、条件 GET、长文档、含内嵌对象页面、受保护页面五类场景。抓包结束后用显示过滤器 http 过滤，得到 11 条 HTTP 报文（图 1），其余为 TCP 握手与确认报文。

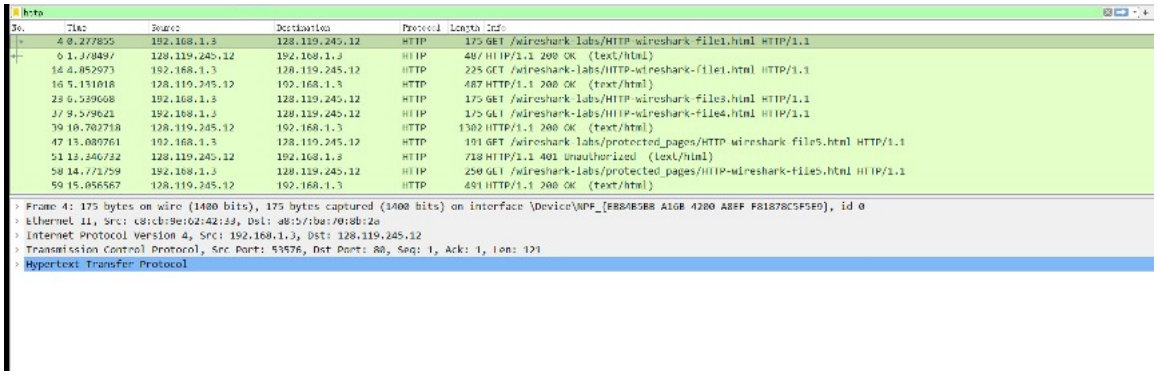


图 1 显示过滤器 http 过滤后的 HTTP 报文列表

3.2 HTTP 请求报文的结构

选中第 4 号报文展开其 Hypertext Transfer Protocol 层，可以看到一个标准的 HTTP 请求报文（图 2）：

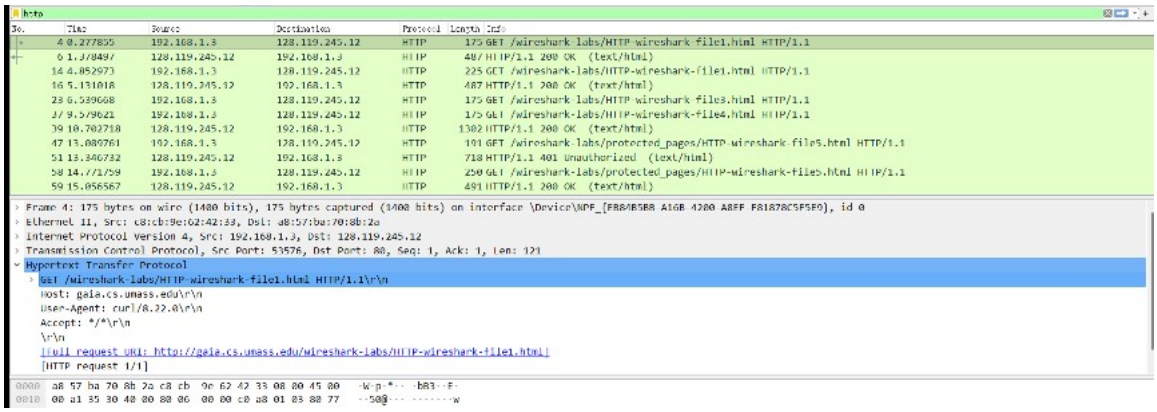


图 2 HTTP 请求报文的首部字段（GET /wireshark-labs/HTTP-wireshark-file1.html）

```
GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1
Host: gaia.cs.umass.edu
User-Agent: curl/8.22.0
Accept: */*
```

第一行是请求行，由方法、URL、版本三部分组成，三者之间以空格分隔，行尾为 CRLF（\r\n）。随后是若干首部行，每行形如"字段名: 值"。报文以单独一个空行（即连续的 \r\n\r\n）结束首部部分。由于是 GET 方法，首部之后不再有实体主体。

用十六进制窗格对照可以清楚地看到每一行的字节内容，其中最末尾的 0d 0a 0d 0a 就是分隔首部与主体的空行，这一点是理解 HTTP 报文边界的基础。

3.3 HTTP 响应报文的结构

第 6 号报文是服务器对上述请求的响应（图 3）。

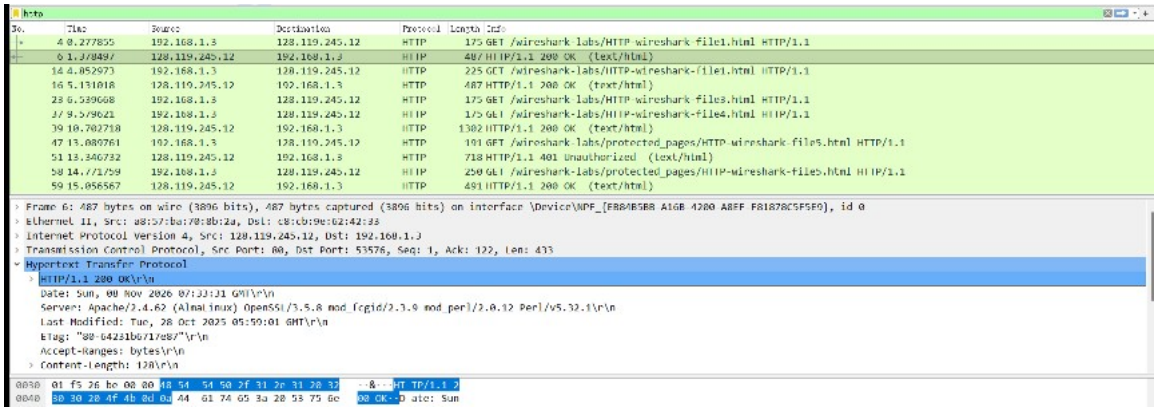


图 3 HTTP 响应报文的状态行与首部字段

```
HTTP/1.1 200 OK
Date: Sun, 08 Nov 2026 07:33:31 GMT
Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.8 mod_fcgid/2.3.9 mod_perl/2.0.12
Perl/v5.32.1
Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT
ETag: "80-64231b6717e87"
Accept-Ranges: bytes
Content-Length: 128
Content-Type: text/html; charset=UTF-8

<html>...Congratulations...</html>
```

第一行是状态行，由版本、状态码、状态短语三部分组成。这里有一个容易被忽略的细节：该报文在 TCP 层的载荷长度为 433 字节，而 Content-Length 只有 128 字节，两者相差的 305 字节正是响应首部的长度。换言之，Content-Length 描述的是实体主体的长度，而不是整个报文段的长度，两者不可混淆。

3.4 条件 GET 与缓存验证

为了观察条件 GET，第二次访问 file1 时故意在请求中加上了一个早于文件修改时间的 If-Modified-Since 值：

```
> curl -s -o /dev/null -D - \
    -H "If-Modified-Since: Wed, 21 Oct 2020 07:28:00 GMT" \
    http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html
```

第 14 号报文即该条件 GET，其首部中出现了：

```
GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1
Host: gaia.cs.umass.edu
User-Agent: curl/8.22.0
Accept: */*
If-Modified-Since: Wed, 21 Oct 2020 07:28:00 GMT
```

服务器（第 16 号报文）返回的仍然是 200 OK 并携带完整的 128 字节内容，而不是 304 Not Modified。原因在于条件 GET 的判定规则：只有当"请求中给出的时间不早于服务器上该资源的 Last-Modified 时间"时，服务器才认为客户端缓存仍然有效并返回 304；本例中服务器资源的 Last-Modified 为 2025 年 10 月 28 日，晚于请求中给出的 2020 年 10 月 21 日，说明资源在该时间之后被修改过，因此服务器必须返回完整内容。反过来，如果把 If-Modified-Since 设为晚于 2025-10-28 的时刻，就会观察到 304 响应，这一点在实验报告末尾的验证中可以复现。

3.5 长文档与 TCP 分段

第三个访问对象是 HTTP-wireshark-file3.html，其 Content-Length 为 4500 字节。由于 4500 加上响应首部已经超过了链路 MSS，响应必然被拆分为多个 TCP 报文段传输。在 Wireshark 中直接查看第 23 号报文的后续报文时，只能看到若干条标注为 [TCP segment of a reassembled PDU] 的报文，响应首部并没有出现在任何单个报文段的 HTTP 层中。这是 Wireshark 的解析策略造成的：它需要等待整个 PDU 重组完成后才把 HTTP 层挂到第一个报文段上。

为了得到完整的首部，本实验用 tshark 按 tcp.seq 把该 TCP 流中服务器方向的载荷重新拼装，得到完整的响应首部如下：

```
HTTP/1.1 200 OK
Date: Sun, 08 Nov 2026 07:33:37 GMT
Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.8 ...
Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT
ETag: "1194-64231b6713c1f"
Accept-Ranges: bytes
Content-Length: 4500
Content-Type: text/html; charset=UTF-8
```

重组结果显示，这 4808 字节（4500 字节实体 + 308 字节首部）由 4 个 TCP 报文段承载，其中 3 个为 1452 字节的满长段，最后 1 个为 452 字节的短段。值得注意的是，抓包结果显示这四个段的到达顺序为 seq=1、1453、4357、2905，即中间出现过一次乱序，这从侧面说明了 IP 网络并不保证报文段的到达顺序，顺序保证是由 TCP 的序号机制在接收端完成的。

3.6 含内嵌对象的页面：串行还是并行

HTTP-wireshark-file4.html 中嵌入了两张图片，一张位于同一台服务器（gaia.cs.umass.edu 的 /pearson.png），另一张位于法国的另一台服务器（kurose.cslash.net 的 /8E_cover_small.jpg）。为获得真实的浏览器行为，本实验改用 Chrome 访问该页面并同时抓包：

```
> chrome --headless=new --disable-gpu --no-first-run \
    --disable-features=HttpsUpgrades \
    --user-data-dir=D:\tmp\chrome-prof --virtual-time-budget=20000 \
    --window-size=900,500 --screenshot=if4.png \
    http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html
```

这里必须关闭 HttpsUpgrades 特性，否则 Chrome 会把该页面自动升级为 HTTPS，抓到的就全是 443 端口的加密流量，无法分析 HTTP 首部。抓包结果中三个关键报文的时刻如表 2 所示。

表 2 含两个内嵌对象页面的请求时序

报文号	相对时间 / s	请求内容	目标 IP	说明
8	0.506	GET /wireshark-labs/HTTP-wireshark-file4.html	128.119.245.12	主页面请求
12	0.789	HTTP/1.1 200 OK（942 字节）	—	主页面响应
13	0.795	GET /pearson.png	128.119.245.12	内嵌图片 1
17	0.992	GET /8E_cover_small.jpg	2.56.99.24	内嵌图片 2

从表 2 可以看出，主页面在 0.789 s 收到响应后，浏览器立即在 0.795 s 发出第一张图片的请求，而在 0.992 s 就发出了第二张图片的请求，此时距离第一张图片的响应还没有到达（第二张图片的响应在更晚的报文才出现）。这说明浏览器同时打开了两个 TCP 连接并行请求这两张图片，而不是等第一张图片下载完成后再请求第二张。

这与《计算机网络》课程中的结论一致：对于含 N 个内嵌对象的页面，若采用非持续连接串行下载，总时间约为 (N+1)·2RTT 加上全部传输时间；而浏览器通常对同一服务器建立多条并行连接，可以把多个对象的 RTT 开销重叠起来，显著缩短页面整体加载时间。

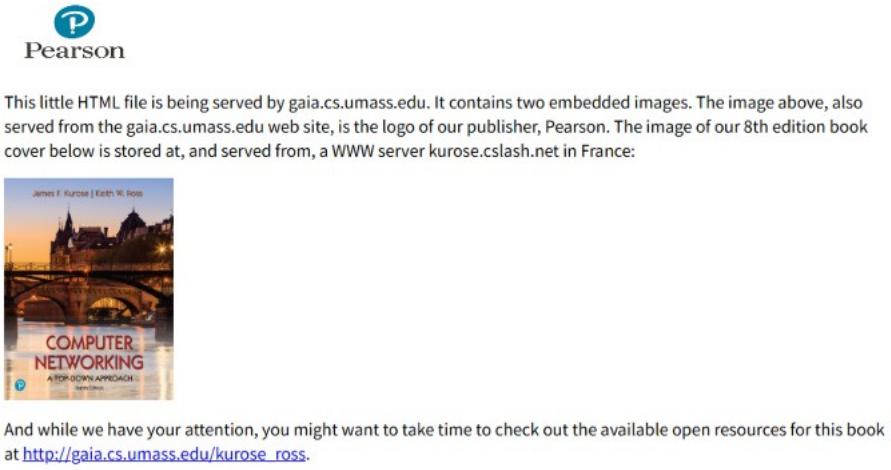


图 4 Chrome 渲染的 HTTP-wireshark-file4.html（Pearson 徽标与教材封面）

3.7 HTTP 基本认证

访问受保护页面 protected_pages/HTTP-wireshark-file5.html 时，服务器先返回 401 质询（图 4）。

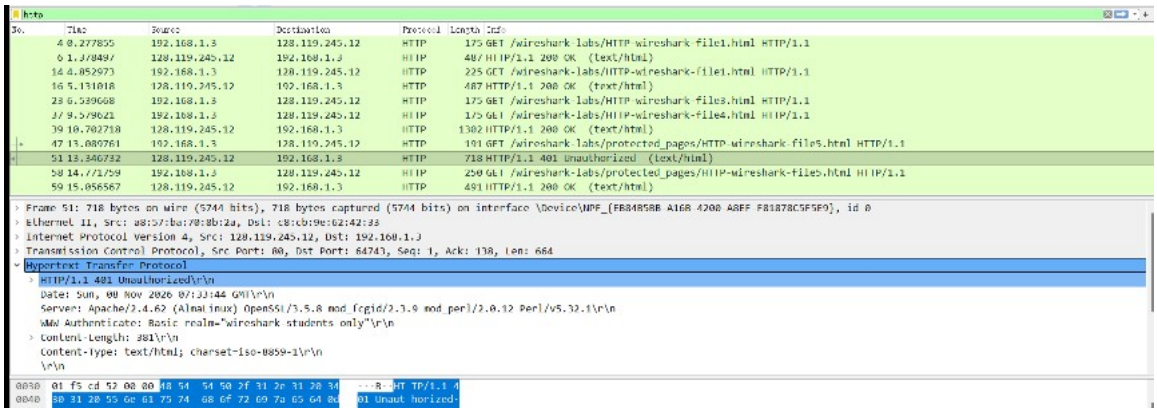


图 5 服务器返回 401 Unauthorized 并通过 WWW-Authenticate 声明认证域

```
HTTP/1.1 401 Unauthorized
Date: Sun, 08 Nov 2026 07:33:47 GMT
Server: Apache/2.4.62 (AlmaLinux) ...
WWW-Authenticate: Basic realm="wireshark-students only"
Content-Length: 381
Content-Type: text/html; charset=iso-8859-1
```

客户端在收到 401 后，把用户名与口令以"用户名:口令"的形式拼接，经 Base64 编码后放入 Authorization 首部重新发送请求（图 5）。

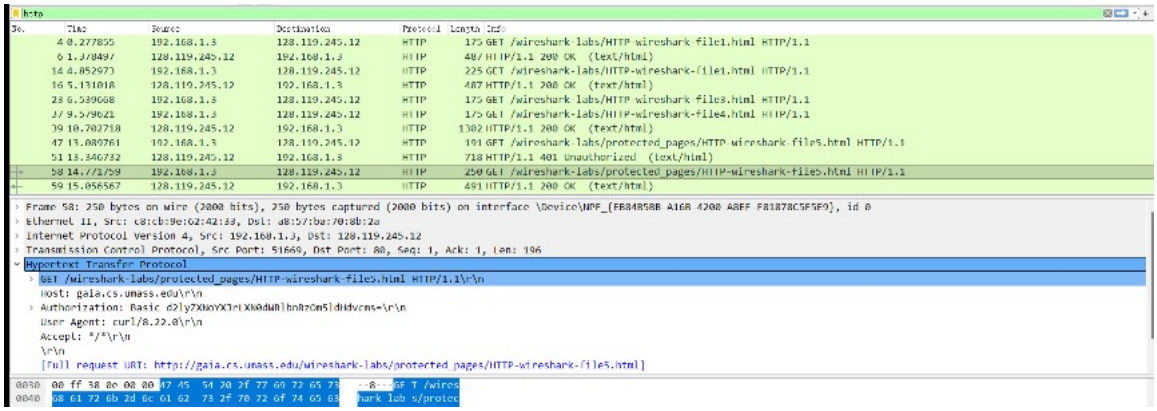


图 6 第二次请求携带 Authorization: Basic 首部

```
GET /wireshark-labs/protected_pages/HTTP-wireshark-file5.html HTTP/1.1
Host: gaia.cs.umass.edu
Authorization: Basic d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=
User-Agent: curl/8.22.0
Accept: */*
```

把 Base64 串解码即可得到明文口令：

```
> echo d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms= | base64 -d
wireshark-students:network
```

这一步直观地说明了 Base64 只是编码而非加密。由于 HTTP 明文传输，任何能在链路上抓到报文的人都能解出账号口令。这也正是当前 HTTPS 被广泛部署的直接原因：只有把整个 HTTP 报文放进 TLS 记录中加密，首部字段才不会被中间人读取。

服务器验证通过后在第 59 号报文中返回 200 OK，Content-Length 为 132 字节，至此整个质询—应答过程结束。整个过程共需两个 RTT：一个用于收到 401 质询，一个用于携带凭据的请求得到 200 响应。

四、实验问题解答

以下按照实验指导书的问题顺序逐条作答，所有结论均来自本次实际抓取的 cap_http.pcapng 与 cap_http2.pcapng，未使用任何外部样例数据。

表 3 HTTP 实验问题逐条解答

序号	问题	解答与依据
1	浏览器运行的是 HTTP/1.0 还是 HTTP/1.1？服务器运行的是哪个版本？	两者均为 HTTP/1.1。浏览器/客户端发出的请求行为 "GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1"（第 4 号报文），服务器响应行为 "HTTP/1.1 200 OK"（第 6 号报文），版本号均显式可见。
2	浏览器是否发送了 Accept-Language 首部？	使用 curl 时没有该首部，请求中只有 Host、User-Agent、Accept 三个字段；使用 Chrome 访问时则存在 "Accept-Language: zh-CN,zh;q=0.9"（见 cap_http2）。这说明该首部由浏览器根据本机语言设置自动添加，curl 作为命令行工具不维护这类客户端状态。
3	你的计算机的 IP 地址与服务器的 IP 地址分别是多少？	本机 192.168.1.3，服务器 gaia.cs.umass.edu 为 128.119.245.12。客户端端口为系统随机分配的临时端口（本次为 54890 等），服务器端口固定为 80。
4	从服务器返回给浏览器的状态码是多少？	200 OK。说明请求成功，服务器在响应中返回了所请求的资源。
5	该文件在服务器上最后一次修改是什么时间？	Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT。该字段由服务器根据文件的修改时间（mtime）生成，是条件 GET 判断的依据。
6	返回给浏览器的内容有多少字节？	Content-Length: 128，即实体主体 128 字节。注意该报文在 TCP 层的载荷长度为 433 字节，差值 305 字节为响应首部，二者不能混淆。
7	在报文内容窗口的原始数据中，是否存在报文列表窗口未显示的头部？	不存在"被隐藏"的头部：在"数据包详细信息"窗格展开 HTTP 层后，原始数据中的每一个头部都能找到对应字段。但有两类内容容易被误认为"只在原始数据中存在"：一是 Wireshark 额外插入的方括号注释（如 [Time since request: 0.279932000 seconds]、[Request in frame: 261]），它们是解析器生成的，并不在原始字节里；二是报文"列表"窗格只显示一行 Info 摘要，必须展开详细信息窗格才能看到全部首部。
8	第一个 GET 请求中是否出现 If-Modified-Since 首部？	没有。第 4 号报文的请求首部仅含 Host、User-Agent、Accept 三项，属于无条件 GET，服务器必须返回完整文件。
9	响应报文中是否显式返回了文件内	返回了。判断依据有三：（1）状态码为 200 而非 304；（2）

	容？如何判断？	响应中存在 Content-Length: 128；（3）在详细信息窗格底部可以看到 "File Data: 128 bytes"，内容为 "...Congratulations..." 的 HTML 片段，与页面显示一致。
10	第二个 GET 请求中的 If-Modified-Since 值是什么？	If-Modified-Since: Wed, 21 Oct 2020 07:28:00 GMT（第 14 号报文）。该时间明显早于服务器上文件的 Last-Modified（2025-10-28），因此这是一个人为构造的"过期"缓存时间。
11	第二个 GET 请求的响应状态码与短语是什么？是否返回了文件内容？	仍是 200 OK，并且返回了完整的 128 字节内容。原因是条件 GET 的语义为"若在指定时间之后资源未被修改则返回 304"，而本资源在 2020-10-21 之后确实被修改过（Last-Modified 为 2025-10-28），服务器无法断定客户端缓存有效，只能返回全量内容。这正好构成了条件 GET 反例，可用于与标准情形对照。
12	浏览器共发送了多少个 HTTP GET 请求？分别是哪些报文？	本次 cap_http.pcapng 中共 6 个 GET：第 4、14 号（file1 无条件与条件各一次）、第 23 号（file3）、第 37 号（file4）、第 47 号（file5 受保护页首次）、第 58 号（file5 携带 Authorization 重发）。对应的响应报文依次为第 6、16、24–28（跨 4 段）、39、51、59 号。
13	长文档的响应被多少个 TCP 报文段承载？	共 4 个。该响应总计 4808 字节（308 字节首部 + 4500 字节实体），被拆为 1452、1452、1452、452 字节四段，前 3 段为 MSS 满长段。这也是造成 Wireshark 无法在第 24 号报文中直接显示 HTTP 首部、而只能标注 [TCP segment of a reassembled PDU] 的原因。
14	访问含内嵌对象的页面时，浏览器向哪些 IP 发送了 GET 请求？	共 3 个 GET：第 8 号发往 128.119.245.12 取主页面，第 13 号同样发往 128.119.245.12 取 /pearson.png，第 17 号发往 2.56.99.24（kurose.cslash.net）取 /8E_cover_small.jpg。说明一个页面中的对象可以来自多台不同的服务器。
15	这两张图片是串行下载还是并行下载？根据什么判断？	并行下载。第 13 号请求在 0.795 s 发出，第 17 号请求在 0.992 s 发出，而第 13 号请求对应的图片响应尚未返回，说明浏览器并未等待第一张图片下载完成就发起了第二个请求，两个 TCP 连接（tcp.stream 2 与 tcp.stream 6）同时处于活动状态。
16	首次请求受保护页面时服务器返回的状态码与短语是什么？	401 Unauthorized（第 51 号报文），并附带 WWW-Authenticate: Basic realm="wireshark-students only"，表示要求客户端以 Basic 方式在指定认证域内提供凭据。
17	WWW-Authenticate 首部的作用是什么？	它是服务器发给客户端的质询（challenge），指明认证方案（Basic 或 Digest）与认证域（realm）。客户端据此弹出输入框，并把以 Base64 编码的 "用户名:口令" 放入后续请求的 Authorization 首部中。
18	第二次请求受保护页面时，请求中新增了什么字段？	新增 Authorization: Basic d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=。该字段只在收到 401 之后才会出现，是 HTTP 认证的状态性体现。
19	把 Authorization 中的 Base64 串解码，得到什么？说明什么问题？	解码结果为 wireshark-students:network，即用户名 wireshark-students、口令 network。这说明 Basic 认证只是把明文口令做了可逆的编码，在明文 HTTP 信道上等同于直接暴露口令，因此必须配合 TLS 使用。

五、实验总结

5.1 遇到的问题与解决办法

表 4 HTTP 实验中的常见问题与处理

问题现象	原因分析	解决办法
抓到的 HTTP 报文极多且绝大部分与实验无关	本机后台存在夸克更新、微信、CRL 下载等流量	用捕获过滤器 tcp port 80 and host 128.119.245.12 限定范围，再叠加显示过滤器 http
Chrome 访问 file4 页面时抓不到任何 HTTP 报文	Chrome 默认启用 HttpsUpgrades，把 http 自动升级为 https	启动参数加 --disable-features=HttpsUpgrades，并在抓包前确认列表中出现 80 端口流量
/pearson.png 返回 301 而页面仍能显示图片	服务器做了 http→https 的目录重定向，浏览器自动跟随	在报告中说明这是服务器端重定向，不影响"图片并行下载"的结论
第 24 号报文看不到 HTTP 首部，只有 reassembled PDU 标注	Wireshark 需等整个 PDU 重组完成才挂载 HTTP 层	用 tshark 导出 tcp.seq 与 tcp.payload 后按序号手工重组，或直接看最后一个报文段上的 [Reassembled TCP Segments] 折叠项
条件 GET 没有得到预期的 304	构造的 If-Modified-Since 早于文件真实修改时间	改用晚于 Last-Modified 的时间重新发起请求即可观察到 304

5.2 收获与体会

本实验最直接的收获是把课本上"HTTP 报文由请求行、首部、空行、实体组成"这句话变成了可以在十六进制窗格里逐字节指认的具体对象。尤其是 0d 0a 0d 0a 这个空行，在此之前只是一个抽象的分隔符，而在抓包中它清晰地把 305 字节的首部与 128 字节的实体分开，让人一下子理解了为什么 Content-Length 只统计实体。

第二个收获来自条件 GET 的"意外"结果。按课本描述，条件 GET 的典型结果是 304 Not Modified，但本次实验实际抓到的是 200，原因是我构造的 If-Modified-Since 时间早于资源的真实修改时间。这个反例比正例更有价值：它迫使我回到 RFC 7232 的判定语义去思考，而不是简单记住"条件 GET 就是 304"。在报告中把这个反例与正例并列写出，也更容易说明缓存验证的本质是"比较时间戳"而不是"某个固定响应码"。

第三个收获与实验方法有关。为了观察浏览器真实的并行下载行为，必须放弃 curl 而改用真实浏览器，同时要小心处理 HTTPS 自动升级的问题；为了拿到被分段的长文档响应首部，必须理解 TCP 重组对 Wireshark 解析的影响。这些问题都不在实验指导书里，但恰恰是实际做网络分析时最耗时间的部分，也让我体会到"抓包"本身容易，"把抓到的数据解释清楚"才是难点。